

Week 8 Live Coding

Three polls, three answers

The phone poll says Rep +4. The online poll says Dem +8. They surveyed different people. Today we'll see why they disagree and what happens when we reweight.

Five things we will do:

1. Compute unweighted topline for both polls
2. Check how much of the 12-point gap sampling noise could explain
3. Compare the age distribution of each sample to the population
4. Reweight the phone poll by age (step by step), and price what the weighting cost
5. Reweight the online poll and compare

Before you start

Save your own copy first. Go to **File → Save a copy in Drive**. A new tab opens with your own copy. Work in that tab; edits to the original are not saved.

The data loads itself. There is nothing to download or upload. The setup cell below pulls the data straight from the course repository; you just need to be online.

Stuck? See the Colab troubleshooting guide on the syllabus.

Setup

Run the cell below to load the data.

```
In [1]: import pandas as pd
import numpy as np

phone = pd.read_csv('https://raw.githubusercontent.com/joshuakalla/
                    'main/weeks/wk08_trusting_polls/data/phone_poll
online = pd.read_csv('https://raw.githubusercontent.com/joshuakalla/
                    'main/weeks/wk08_trusting_polls/data/online_po

# The population's true age distribution (from the voter file)
pop_age = {'18-29': 0.180, '30-44': 0.250, '45-64': 0.321, '65+': 0

print(f'Phone poll: {len(phone):,} respondents')
print(f'Online poll: {len(online):,} respondents')
```

Phone poll: 1,001 respondents
Online poll: 1,002 respondents

```
In [2]: phone.head()
```

```
Out[2]:
```

	respondent_id	age_group	vote_intent	poll
0	1	45-64	dem	phone_poll
1	2	30-44	dem	phone_poll
2	3	65+	rep	phone_poll
3	4	30-44	undecided	phone_poll
4	5	65+	dem	phone_poll

Part 1: Unweighted topline

The simplest thing: what fraction of each poll's respondents said Dem vs. Rep?

The cells below also print the **lead**: one candidate's share minus the other's. That is what "Rep +4" and "Dem +8" in the case mean. A lead is a different number from either candidate's own support, and it will turn out to have its own, larger uncertainty.

```
In [3]: # Phone poll topline
phone_topline = phone['vote_intent'].value_counts(normalize=True).sort_index()
print('Phone poll (unweighted):')
print(phone_topline.round(3))
print(f'Dem lead: {100*(phone_topline.get("dem",0) - phone_topline.get("rep",0))} pp')
```

```
Phone poll (unweighted):
vote_intent
dem          0.462
rep          0.500
undecided    0.038
Name: proportion, dtype: float64
Dem lead: -3.9 pp
```

```
In [4]: # Online poll topline
online_topline = online['vote_intent'].value_counts(normalize=True).sort_index()
print('Online poll (unweighted):')
print(online_topline.round(3))
print(f'Dem lead: {100*(online_topline.get("dem",0) - online_topline.get("rep",0))} pp')
```

```
Online poll (unweighted):
vote_intent
dem          0.509
rep          0.433
undecided    0.058
Name: proportion, dtype: float64
Dem lead: +7.6 pp
```

The phone poll shows Rep +4. The online poll shows Dem +8. A 12-point gap!

These are the same race, at the same time. The difference is **who responded**.

Part 2: How much of the 12 points is just sampling noise?

Before blaming bias, ask the Week-5 question: could a 12-point gap happen by **random sampling** alone? Each poll's **margin of error** is just the confidence interval from Week 5, now on a poll number.

```
In [5]: def poll_uncertainty(d, name):
        n = len(d)
        tl = d['vote_intent'].value_counts(normalize=True)
        p_dem, p_rep = tl.get('dem', 0), tl.get('rep', 0)
        lead = p_dem - p_rep
        moe_each = 1.96*np.sqrt(0.25/n) # MoE on one number
        se_lead = np.sqrt((p_dem + p_rep - lead**2)/n) # SE of the lead
        lo, hi = lead - 1.96*moe_each, lead + 1.96*moe_each
        verdict = 'statistical TIE (interval includes 0)' if lo < 0 < hi else 'real lead'
        print(f'{name}: n={n:,}, lead {100*lead:+.1f} pp')
        print(f'    margin of error on each number: ±{100*moe_each:.1f} pp')
        print(f'    95% CI on the lead: [{100*lo:+.1f}, {100*hi:+.1f}] pp')
        print()

        poll_uncertainty(phone, 'Phone poll ')
        poll_uncertainty(online, 'Online poll')
```

```
Phone poll : n=1,001, lead -3.9 pp
    margin of error on each number: ±3.1 pp
    95% CI on the lead: [-10.0, +2.2] pp -> statistical
TIE (interval includes 0)
```

```
Online poll: n=1,002, lead +7.6 pp
    margin of error on each number: ±3.1 pp
    95% CI on the lead: [+1.6, +13.6] pp -> a real lead
```

Read those numbers. Each poll's margin of error (on a single candidate) is about **±3 points**. The uncertainty on the *lead* (the difference between the two candidates) is bigger, about **±6 points**.

Why almost exactly double? Every standard error in this course has the same shape: how much people differ from each other, divided by the square root of n . Score each respondent +1 for the Democrat, -1 for the Republican, 0 for undecided; the lead is the average of that score. A 0/1 answer has people differing by about 0.5; a -1/0/+1 answer has them differing by about 1.0. Same n , twice the variation, twice the standard error.

Two honest caveats. Undecideds score 0, which pulls the -1/0/+1 variation

below 1.0, so the ratio here is just under 2 rather than exactly 2. And "never more than double" is only true against the conventional ± 3 , which assumes a 50–50 race; in a lopsided race the lead's margin can exceed twice a candidate's own margin.

So at $n \approx 1,000$:

- the phone poll's **Rep +4** lead's interval includes 0. A **statistical tie**, not a real Republican lead.
- the online poll's **Dem +8** clears ± 6 . A real lead, but only *barely*: its interval runs from **+1.6** to **+13.6**, so even this poll is consistent with a 2-point race.

This is the Week-5 significance test, now on a **descriptive** quantity (a poll margin): is the lead distinguishable from zero?

The two polls disagree by **11.5 points** (the "12" in the case is $8 - (-4)$ off the rounded topline). That is far more than either poll's ± 6 , so the gap is **bias**: who responds to a phone call vs. an online panel.

But notice we just did the thing this course keeps warning about — eyeballed a difference against the wrong yardstick. A *difference between two polls* has its own standard error, bigger than either poll's, because both numbers wobble. Run it properly in the next cell.

```
In [6]: # Is the 12-point gap real? Test the DIFFERENCE, which has its own SE.
def lead_and_se(d):
    n = len(d)
    p_dem = (d['vote_intent'] == 'dem').mean()
    p_rep = (d['vote_intent'] == 'rep').mean()
    lead = p_dem - p_rep
    return lead, np.sqrt((p_dem + p_rep - lead**2) / n)

lead_ph, se_ph = lead_and_se(phone)
lead_on, se_on = lead_and_se(online)

diff = lead_on - lead_ph
se_diff = np.sqrt(se_ph**2 + se_on**2) # two wobbles, so they add
lo, hi = diff - 1.96*se_diff, diff + 1.96*se_diff

print(f'Difference in the two leads: {100*diff:.1f} pp')
print(f'SE of that difference: {100*se_diff:.1f} pp (bigger than either poll's 3.1)')
print(f'95% interval: [{100*lo:.1f}, {100*hi:.1f}] pp')
```

```
Difference in the two leads: 11.5 pp
SE of that difference: 4.4 pp (bigger than either poll's 3.1)
95% interval: [+3.0, +20.0] pp
```

The gap is real: the interval does not include zero. But read the lower bound, the way you read the online poll's +1.6. As little as **3 points** of the

disagreement could be sampling luck. Calling it a flat 12 points of bias overstates what this data can support.

Even 3 points is bias, though, and that is the part no margin of error ever claimed to measure.

Part 3: Who responded?

Let's compare the age distribution of each poll's sample to the actual population.

```
In [7]: # Age distribution: sample vs. population
phone_age = phone['age_group'].value_counts(normalize=True).sort_index()
online_age = online['age_group'].value_counts(normalize=True).sort_index()
pop_age_series = pd.Series(pop_age).sort_index()

comparison = pd.DataFrame({
    'Population': pop_age_series,
    'Phone poll': phone_age,
    'Online poll': online_age
})
print(comparison.round(3))

# And how each age group actually votes, in the phone poll
dem_by_age = phone.groupby('age_group')['vote_intent'].apply(lambda x: x.value_counts(normalize=True))
print('\nPhone poll, Dem share within each age group:')
print((100*dem_by_age).round(1))
```

	Population	Phone poll	Online poll
18-29	0.180	0.060	0.333
30-44	0.250	0.170	0.308
45-64	0.321	0.359	0.245
65+	0.249	0.412	0.114

Phone poll, Dem share within each age group:

age_group

18-29 58.3

30-44 50.6

45-64 46.2

65+ 42.5

Name: vote_intent, dtype: float64

Read that table. The phone poll has 41% of its sample in the 65+ group, but the population is only 25% 65+. The online poll has 33% in the 18-29 group, but the population is only 18% 18-29.

Each poll oversamples the demographic group most likely to respond to its mode:

- Phone → older voters (who pick up the phone) → Republican lean
- Online → younger voters (who are online) → Democratic lean

The topline difference is driven by the sample composition, not by a real 12-point swing.

Part 4: Reweight the phone poll

A note on difficulty. This section introduces a new idea: weighted means. The logic is simple. Give underrepresented groups more weight and overrepresented groups less. There is one new code pattern here (multiplying and dividing group shares). I find weighted means fiddly the first time, so we will build it step by step: one group by hand first, then the general formula.

Step 1: One group by hand

The phone poll has 6% 18-29-year-olds, but the population is 18%. So 18-29-year-olds should count 3× more. The weight for this group is:

$\text{weight} = \text{population share} / \text{sample share} = 0.180 / 0.06 = 3.0$

```
In [8]: # Weight for one group: 18-29 in the phone poll
sample_share_young = phone_age['18-29']
pop_share_young = pop_age['18-29']
weight_young = pop_share_young / sample_share_young

print(f'Sample share (18-29): {sample_share_young:.3f}')
print(f'Population share (18-29): {pop_share_young:.3f}')
print(f'Weight: {weight_young:.2f}')
print(f'Meaning: each young respondent counts {weight_young:.1f}x as much as others')
```

```
Sample share (18-29): 0.060
Population share (18-29): 0.180
Weight: 3.00
Meaning: each young respondent counts 3.0x as much as others
```

Step 2: Weights for all groups

Now compute the weight for every age group. Same formula: population share / sample share.

```
In [9]: # Compute a weight for each respondent based on their age group
# Step 1: build a dictionary of weights
weights_dict = {}
for group in pop_age:
    weights_dict[group] = pop_age[group] / phone_age[group]
    print(f'{group}: pop={pop_age[group]:.3f}, sample={phone_age[group]:.3f}, weight={weights_dict[group]:.2f}')

# Step 2: assign each respondent their group's weight
phone['weight'] = phone['age_group'].map(weights_dict)
phone[['respondent_id', 'age_group', 'vote_intent', 'weight']].head()
```

18-29: pop=0.180, sample=0.060, weight=3.00
 30-44: pop=0.250, sample=0.170, weight=1.47
 45-64: pop=0.321, sample=0.359, weight=0.90
 65+: pop=0.249, sample=0.412, weight=0.60

Out[9]:

	respondent_id	age_group	vote_intent	weight
0	1	45-64	dem	0.895045
1	2	30-44	dem	1.472059
2	3	65+	rep	0.604973
3	4	30-44	undecided	1.472059
4	5	65+	dem	0.604973
5	6	65+	undecided	0.604973
6	7	45-64	dem	0.895045
7	8	65+	rep	0.604973
8	9	45-64	rep	0.895045
9	10	45-64	rep	0.895045

Step 3: The reweighted topline

A **weighted mean** gives each respondent's answer a different amount of influence. Young respondents (undersampled) get more weight; old respondents (oversampled) get less.

To compute the weighted Dem share: multiply each respondent's Dem indicator (1 if Dem, 0 otherwise) by their weight, sum up, and divide by the total weight.

```
In [10]: # Weighted Dem share for the phone poll
phone['is_dem'] = (phone['vote_intent'] == 'dem').astype(int)
phone['is_rep'] = (phone['vote_intent'] == 'rep').astype(int)

weighted_dem = (phone['is_dem'] * phone['weight']).sum() / phone['weight'].sum()
weighted_rep = (phone['is_rep'] * phone['weight']).sum() / phone['weight'].sum()

print(f'Phone poll (unweighted): Dem {100*phone_topline.get("dem",0)}, Rep {100*phone_topline.get("rep",0)}%')
print(f'Phone poll (reweighted): Dem {100*weighted_dem:.1f}%, Rep {100*weighted_rep:.1f}%')
print(f'Dem lead shifted from {100*(phone_topline.get("dem",0)-phone_topline.get("rep",0))} to {100*(weighted_dem-weighted_rep)}%')
```

Phone poll (unweighted): Dem 46.2%, Rep 50.0%
 Phone poll (reweighted): Dem 48.6%, Rep 46.6%
 Dem lead shifted from -3.9 to +1.9

Reweighting moved the phone poll from **Rep +4** to **Dem +2**. Correcting the age imbalance removed the entire Republican lean and then some.

Reweighting fixes the *age* imbalance. It does not fix *within-age* nonresponse

bias: the 65-year-olds who pick up phone polls may differ from the 65-year-olds who don't, and no amount of age weighting can reach that. Notice what weighting actually did here. It did not add a single voter under 30. It took the 60 young people who answered and made each one count for three.

What the weighting cost

Weighting is not free. Counting some respondents more and others less makes the estimate bounce around more, exactly as if you had interviewed fewer people. The **effective sample size** is how many people an *unweighted* poll would need to be this precise.

```
In [11]: # Effective sample size: how many interviews this weighted poll is '
w = phone['weight']
n_eff = w.sum()*2 / (w**2).sum()

moe_raw = 1.96*np.sqrt(0.25/len(phone))
moe_wtd = 1.96*np.sqrt(0.25/n_eff)

print(f'Respondents:          {len(phone):,}')
print(f'Effective sample size: {n_eff:,.0f}')
print(f'Margin of error: ±{100*moe_raw:.1f} pp unweighted -> ±{100*
```

```
Respondents:          1,001
Effective sample size: 743
Margin of error: ±3.1 pp unweighted -> ±3.6 pp weighted
```

Weighting on **one** variable turned 1,001 interviews into the precision of about 743, and widened the margin of error from ±3.1 to ±3.6 points. That is the real answer to "why not weight on everything": every variable you add costs precision, and the cells get small enough that one respondent can carry a weight of 10 and move the topline by himself.

Part 5: Reweight the online poll and compare

```
In [12]: # Reweight the online poll the same way
online_age = online['age_group'].value_counts(normalize=True).sort_
online_weights = {g: pop_age[g] / online_age[g] for g in pop_age}
online['weight'] = online['age_group'].map(online_weights)
online['is_dem'] = (online['vote_intent'] == 'dem').astype(int)
online['is_rep'] = (online['vote_intent'] == 'rep').astype(int)

ow_dem = (online['is_dem'] * online['weight']).sum() / online['weig
ow_rep = (online['is_rep'] * online['weight']).sum() / online['weig

print('=== After reweighting by age ===')
print(f'Phone poll: Dem {100*weighted_dem:.1f}%, Rep {100*weighted
print(f'Online poll: Dem {100*ow_dem:.1f}%, Rep {100*ow_rep:.1f}%
print(f'\nReweighting narrowed the gap from 12 points to {abs(100*(
```

=== After reweighting by age ===

Phone poll: Dem 48.6%, Rep 46.6% (was Rep +4)

Online poll: Dem 48.4%, Rep 46.8% (was Dem +8)

Reweighting narrowed the gap from 12 points to 0.3 points

Look at how far the gap fell: from 12 points to about **0.3**. Weighting on age alone reconciled two polls that looked like they were measuring different races, and both landed near the university poll's Dem +1.

That is a tidy result, and it is tidier than real life. This dataset was built so that age explains the disagreement. In a real pair of polls, weighting closes some of the gap and leaves the rest, because the people who answer differ from the people who don't *within* every age group, and there is no column for that.

What you've seen today

- Two polls of the same race that disagree by 12 points because they surveyed different people.
- **Unweighted topline:** just count the responses. Biased by who responds.
- **Post-stratification (reweighting):** adjust the sample to match the population on known demographics. Fixes the biggest source of bias but can't fix within-group nonresponse.
- **Weighted mean:** give underrepresented groups more weight. Formula:
`(value * weight).sum() / weight.sum()`.
- **Effective sample size:** `weight.sum()**2 / (weight**2).sum()`.
Weighting buys accuracy and pays for it in precision.

Lying-with-data tag #7: Cherry-picking polls; house effects; ignoring nonresponse bias.

Next, open `wk08_problem_set.ipynb`.